

FEDERAL UNIVERSITY
OF SANTA CATARINA

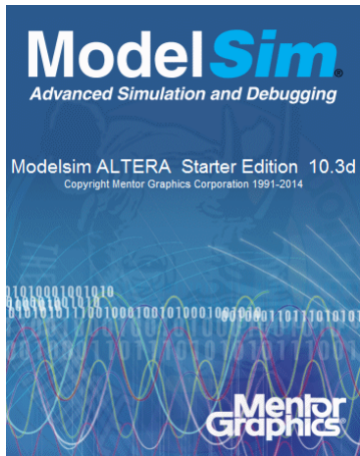
Laboratório 2:
Introdução à VHDL e Modelsim

EEL5105 – Circuitos e Técnicas Digitais

Objetivos

- Apresentar uma visão geral sobre **VHDL** e estudar **exemplos de descrição de hardware** em VHDL
- Familiarização com o simulador utilizado na disciplina ***ModelSim***
- Compreensão do fluxo de projeto de sistemas digitais: Projeto e **simulação**.





Introdução

Tarefa

Introdução

- **VHDL - Visão Geral**

  - **VHDL** é uma linguagem para descrição de hardware
  - **VHDL** = **V**HSIC **H**ardware **D**escription **L**anguage
  - No final da década de 80, **VHDL** se tornou uma linguagem padrão para o **IEEE** (***Institute of Electrical and Electronic Engineers***).
  - Existem diversas ferramentas para simular e sintetizar (gerar hardware) circuitos descritos em **VHDL**.
  - Outras linguagens de descrição de hardware: Verilog, SystemC, AHDL, Handel-C, System Verilog, Abel, Ruby, ...

Introdução

- **VHDL - Visão Geral**

  - O projeto de um circuito digital pode ser descrito em **VHDL** em diversos níveis de abstração (ex.: **estrutural**, **comportamental**).

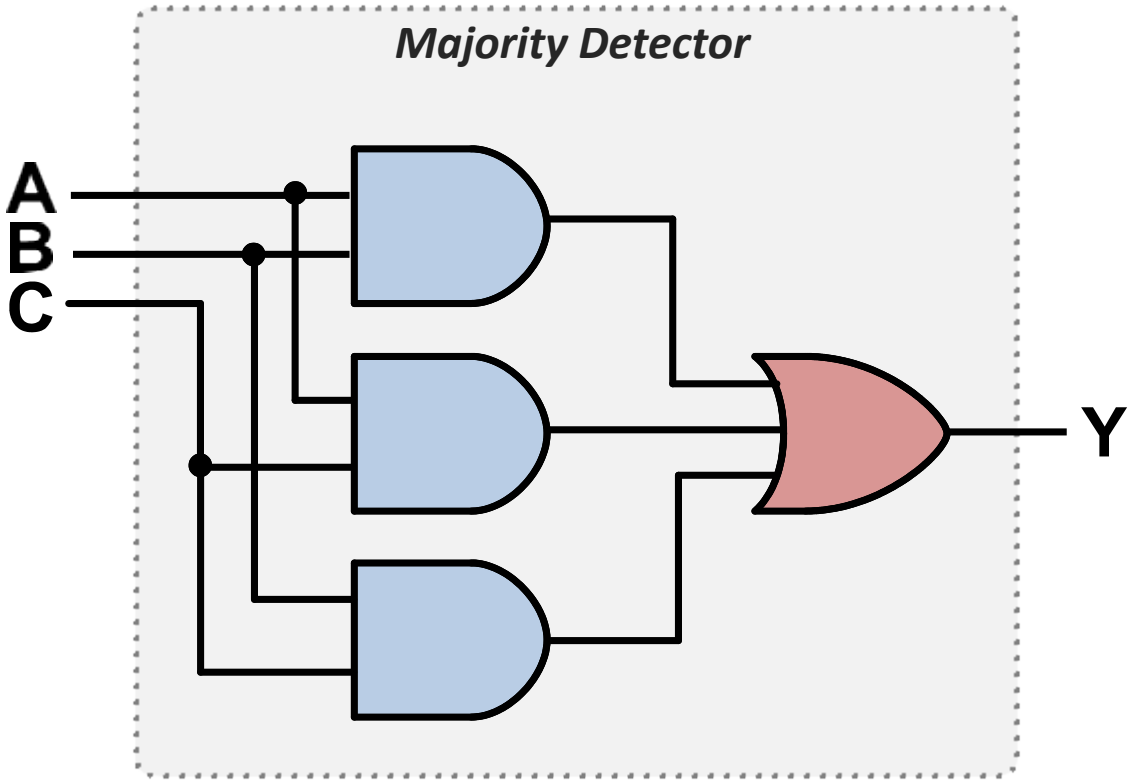
  - Descrições em VHDL podem ser utilizadas para gerar **hardware** (arquivo para configuração de um FPGA, por exemplo).

  - Descrições em VHDL podem ser **simuladas** (executadas em um simulador).

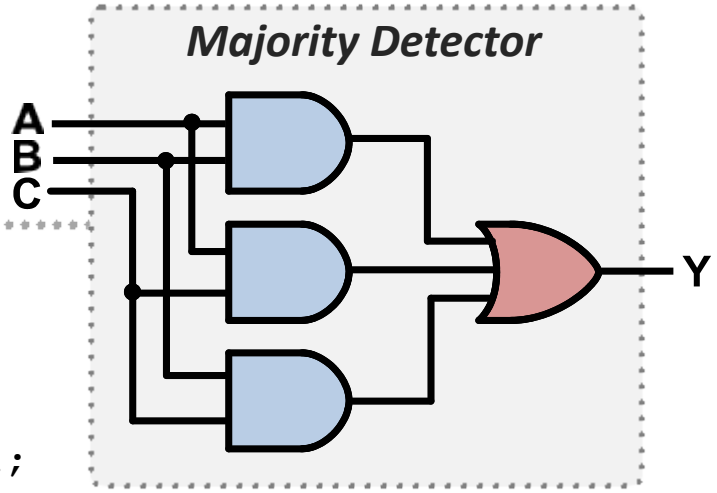
  - A geração de estímulos para simulação VHDL é realizada por intermédio de **testbenches**. Um **testbench** define os estímulos externos a serem utilizados como entrada para o circuito.

Introdução

- **VHDL – Exemplo de código: *Majority Detector***



Introdução



- VHDL – ***Majority Detector***

LIBRARIES

```
library IEEE;
use IEEE.Std_Logic_1164.all;
```

ENTITY

```
entity majority is
port (A: in std_logic;
      B: in std_logic;
      C: in std_logic;
      Y: out std_logic
      );
end majority;
```

ARCHITECTURE

```
architecture circuito_logico of majority is
  signal D,E,F: std_logic;
begin
  Y <= D or E or F;
  D <= A and B;
  E <= A and C;
  F <= B and C;
end circuito_logico;
```

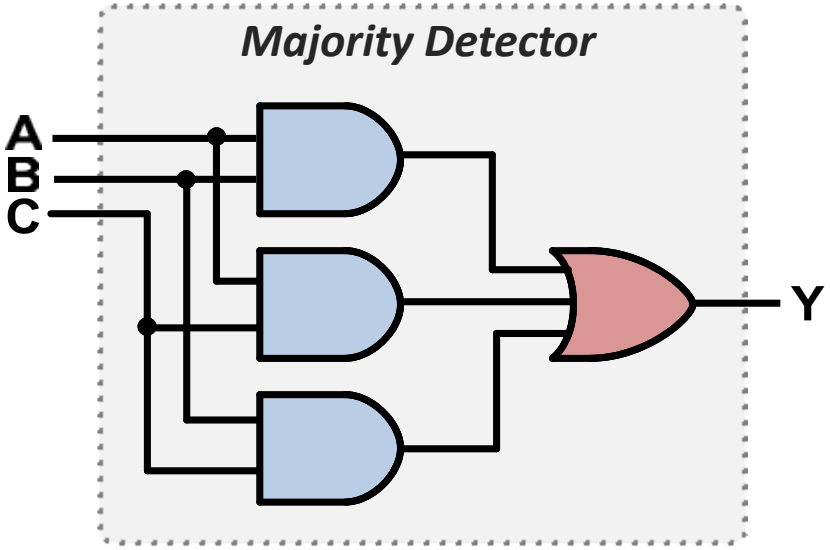
Introdução

- **VHDL – *Majority Detector***
  - **LIBRARIES** : bibliotecas necessárias

```
library IEEE;
use IEEE.Std_Logic_1164.all;
```

Introdução

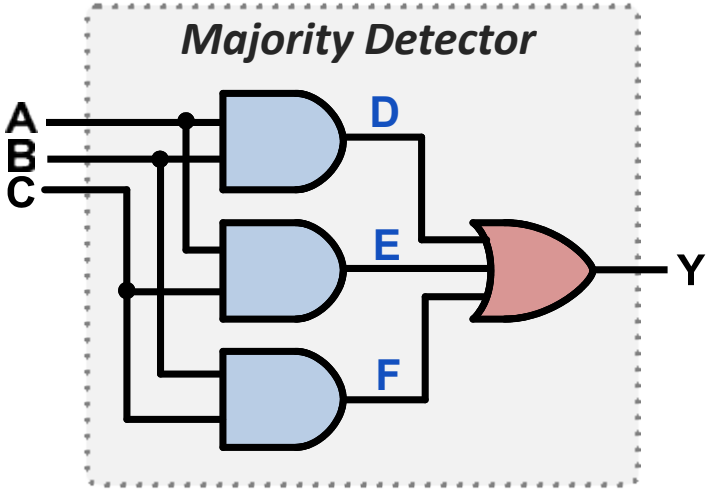
- **VHDL – *Majority Detector***
  - **ENTITY** : define os **pinos** do circuito digital, ou seja, a **interface** entre a lógica implementada e o mundo externo.




```
entity majority is
port (A: in std_logic;
      B: in std_logic;
      C: in std_logic;
      Y: out std_logic
      );
end majority;
```

Introdução

- **VHDL – *Majority Detector***
  - **ARCHITECTURE** : define a funcionalidade do circuito digital, utilizando os **pinos** de entrada e saída listados na **ENTITY**, além de **signals** para fazer as conexões internas.




```
architecture circuito of majority is
  signal D,E,F: std_logic;
begin
  Y <= D or E or F;
  D <= A and B;
  E <= A and C;
  F <= B and C;
end circuito;
```

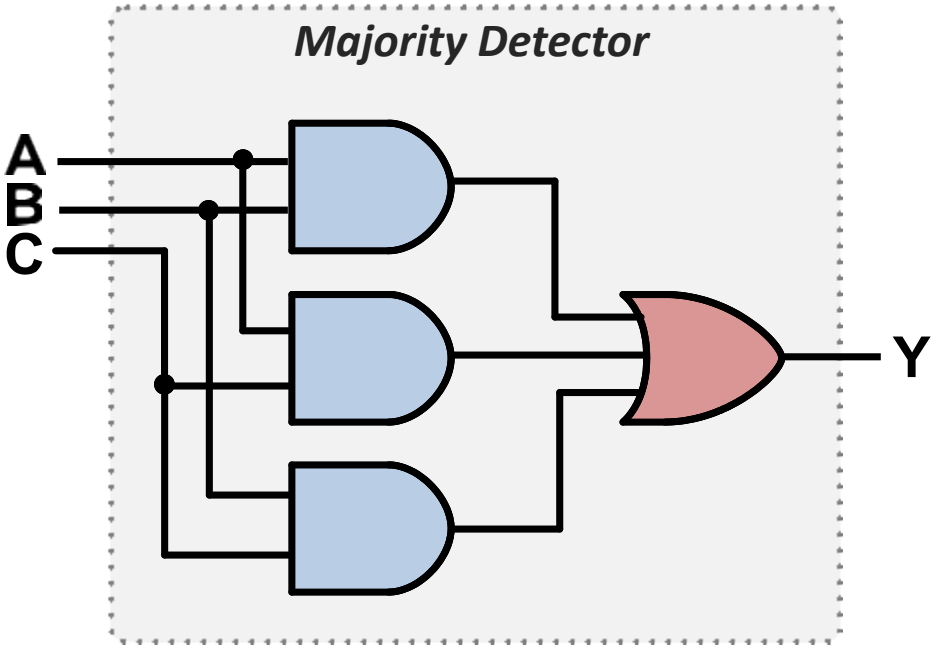
Introdução

Tarefa

Emulação

Tarefa

- Projetar, **simular** e implementar um ***Majority Detector***
  - ***Majority Detector***: saída em nível lógico alto sempre que a maioria dos bits de entrada estiver em nível lógico alto



Y = (A and B) or (A and C) or (B and C)

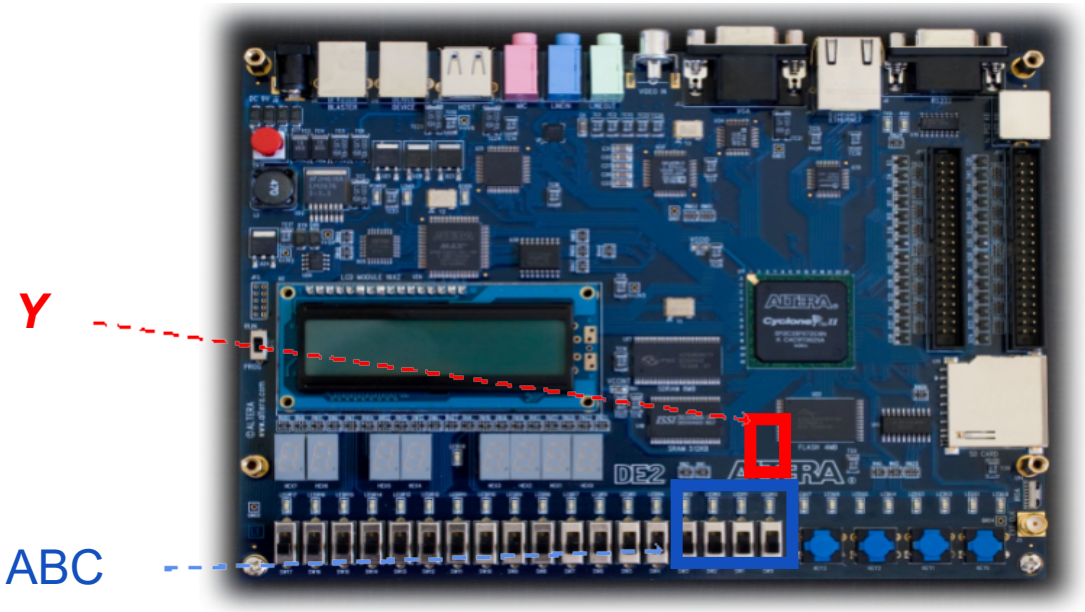
Tabela verdade

A B C	Y
0 0 0	0
0 0 1	0
0 1 0	0
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	1
1 1 1	1

Emulação na placa DE2

Fazer a emulação na placa DE2 do ***Majority Detector***

- Temos de associar **A**, **B**, **C**, e **Y** a **SW(0)**, **SW(1)**, **SW(2)** e **LEDR(0)** na "**`architecture`**" do VHDL. Troque **A** por **SW(0)**, **B** por **SW(1)** e assim por diante.



Tarefa

```
library IEEE;
use IEEE.Std_Logic_1164.all;

entity majority is

port(     SW: in std_logic_vector(2 downto 0);
          LEDR: out std_logic_vector(1 downto 0)
          );
end majority;

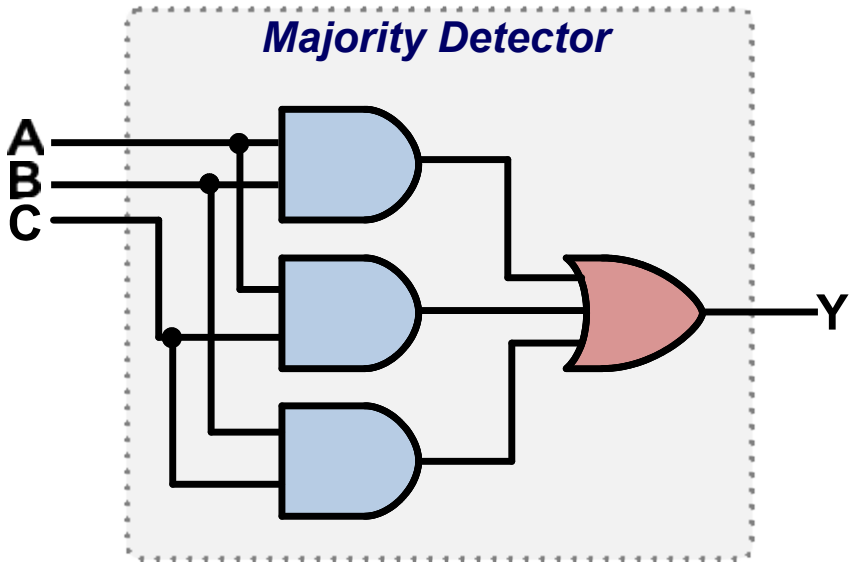
architecture circuito of majority is
signal D, E, F: std_logic;
begin

LEDR(1) <= D or E or F;
D <= SW(0) and SW(1);
E <= SW(0) and SW(2);
F <= SW(1) and SW(2);

end circuito;
```

Tarefa

- Implementar o **Majority Detector** no **Emulador Web**: https://fpgaemu.sites.ufsc.br/login
  - **Atenção:** se o nome da **entity**/**arquivo** não for **usertop**/**usertop.vhd**, é preciso aplicar **Set Top Level** ao arquivo utilizado.
  - Além disso, **ports** precisam ser mapeadas para **chaves** e **leds** do **hardware** ou do **emulador**.



Tarefa

Executar na placa

Para abrir o Quartus:

- **DE1**: Abra um terminal e digite os seguintes comandos:
```
config15
quartus &
```

- **DE2**: Abra um terminal e digite os seguintes comandos:
```
config13
quartus &
```

Note que o menu **File** tem opções diferentes para o Quartus project (**New Project**, **Open Project**, **Close Project**, **Save Project**) e para os arquivos (**New**, **Open**, **Close**, **Save**)

OBS: Quando definir o nomes de pastas, projeto ou arquivos, nunca use caracteres especiais, nem **espaços**, **ç** ou **acentos**.

Seja organizado, salve seu "Quartus project" em uma pasta dentro do Desktop (já seja no seu computador ou no computador do laboratório acessado remotamente).

Tarefa

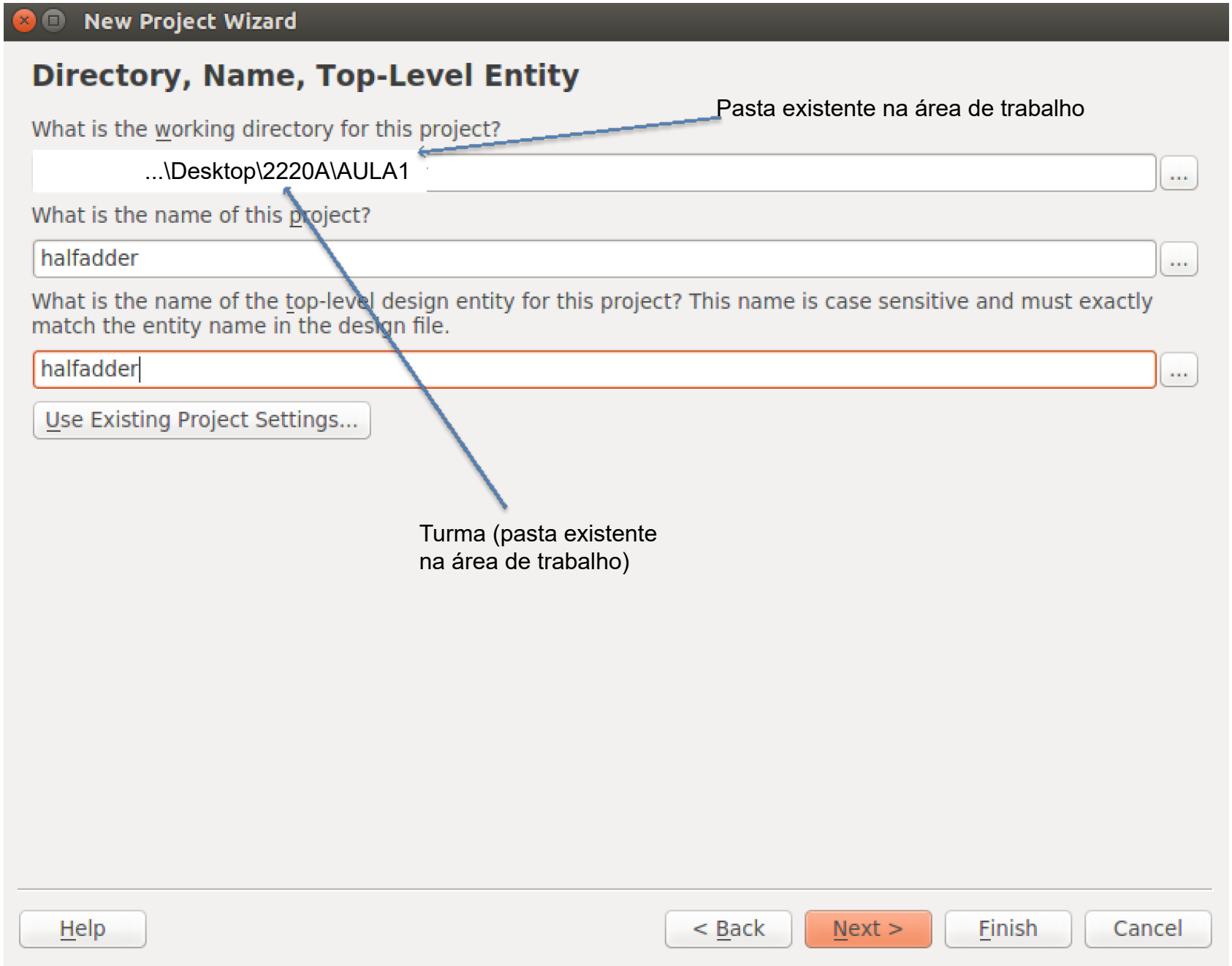
- ***Majority Detector***

  - **Passo 1**: Criar projeto no **Quartus II**
    - Acessar **File -> New Project Wizard** e criar um projeto como feito na aula anterior (**ver próximos slides**)
      - Sugestão de nome do projeto: **majority**

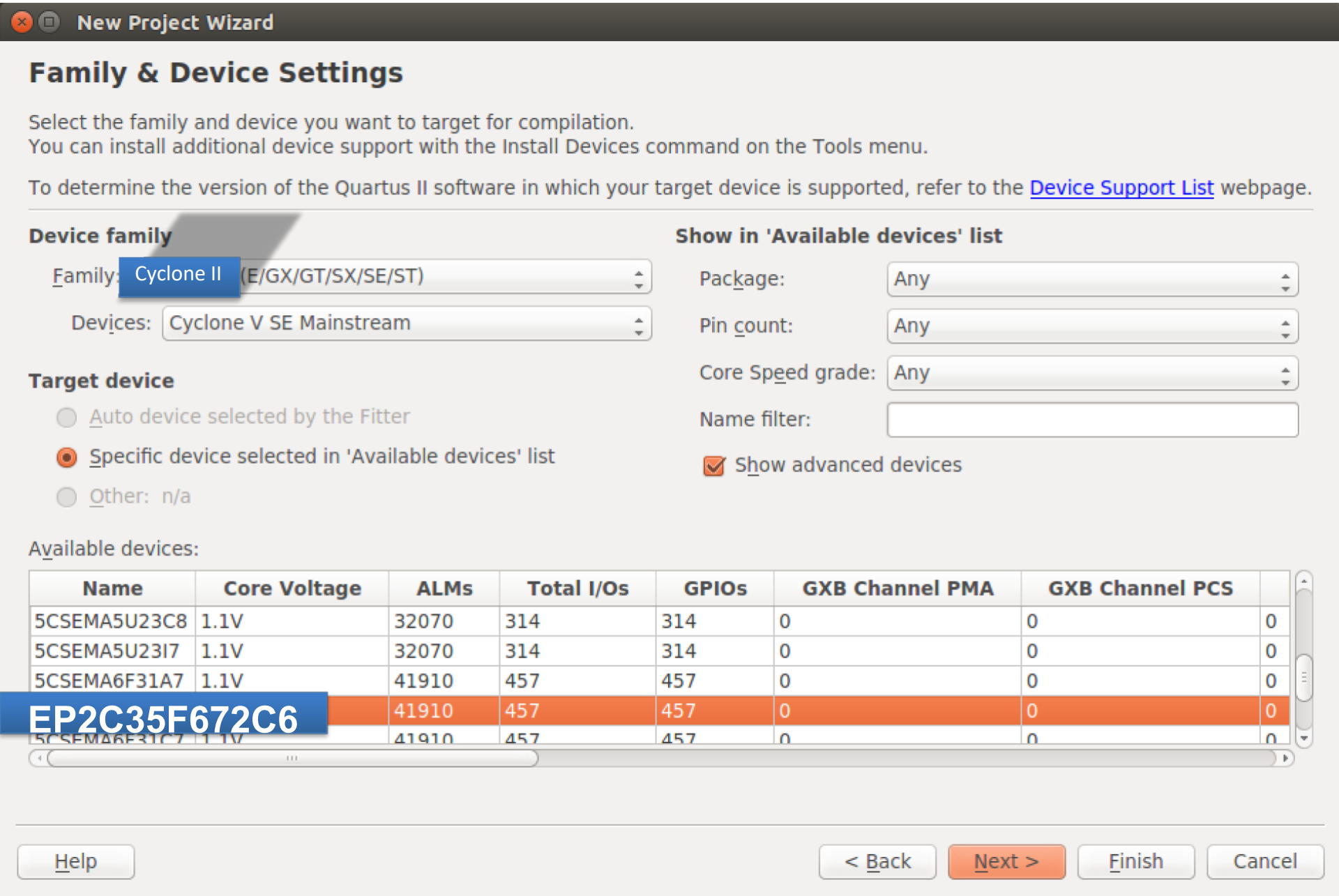
  - **Passo 2**: Criar arquivo do tipo **VHDL** dentro do projeto.
    - **File -> New -> Design Files -> VHDL File**
    - **File -> Save As**: ***Majority.vhd***
    - Com arquivo aberto: **Project -> Set As Top Level Entity**

Atenção: nome do arquivo deve ser igual ao da **ENTITY** que você vai criar.

Escolher a pasta e nome do projeto

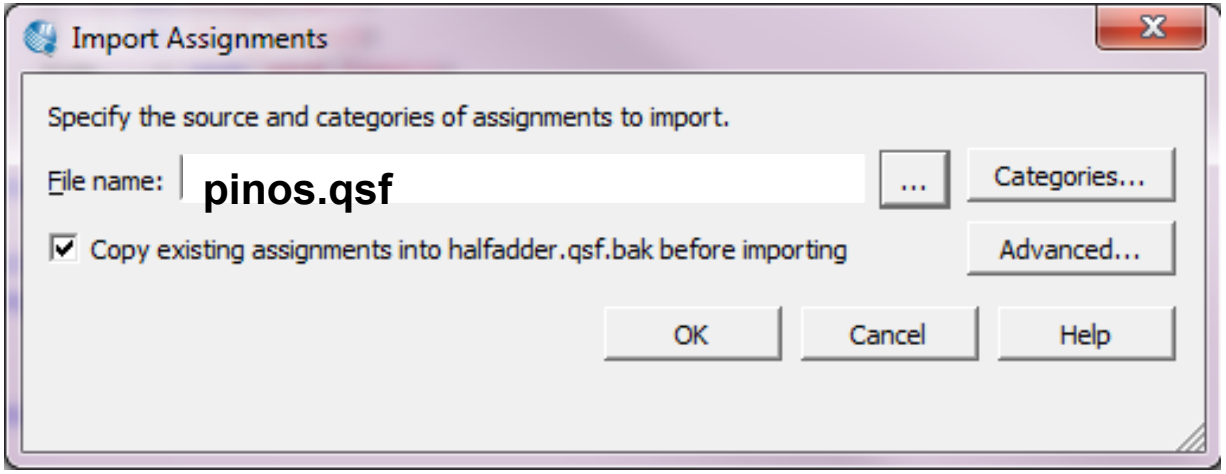


Selecionar o dispositivo alvo (FPGA)



Importar arquivo que seleciona pinos de entrada

- **Assignments -> Import Assigments (procurar no Moodle e usar o arquivo *pinos.qsf*)**



- **Com isso, os pinos do FPGA serão associados às entradas declarada na *entity* do VHDL**

Tarefa

- ***Majority Detector***
  - **Passo 3**: Escreva o código **VHDL** do ***Majority*** no arquivo criado.
    - ***Libraries***:

```
library IEEE;
use IEEE.Std_Logic_1164.all;
```

    - ***Entity***:

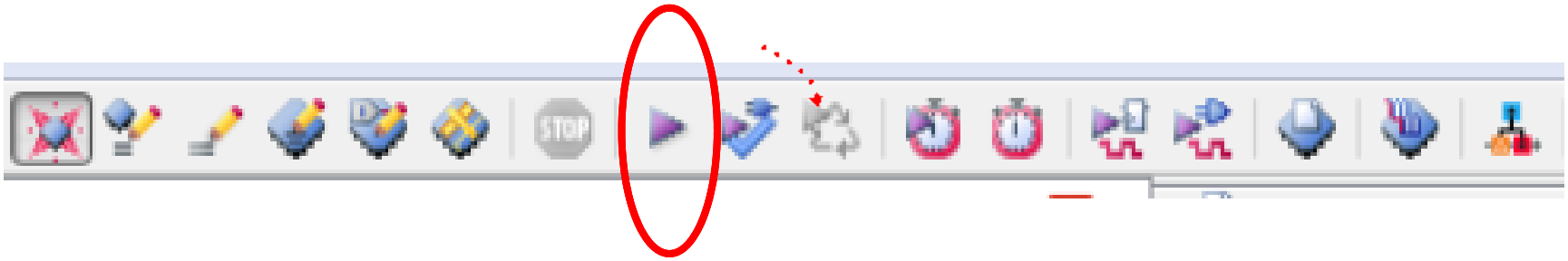
```
entity majority is

port(     SW: in std_logic_vector(2 downto 0);
          LEDR: out std_logic_vector(1 downto 0)
     );
end majority;
```

    - **Escrever a *Architecture***

Tarefa

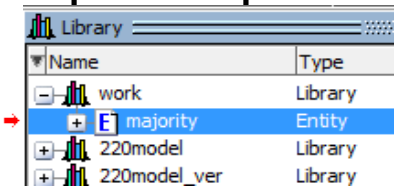
- **Passo 4**: Compilar o projeto.
  - Clique então em **Processing -> Start Compilation** para fazer a síntese do seu projeto ou pressione:



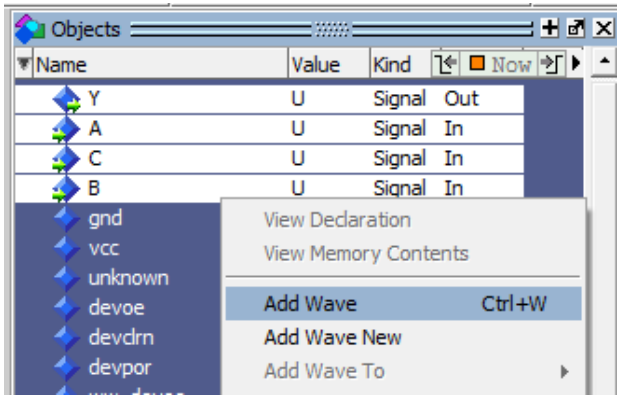
  - Verifique que não possui erros. A ideia dessa aula é se familiarizar com o Quartus e com o processo de compilação e verificação de erros.

Tarefa

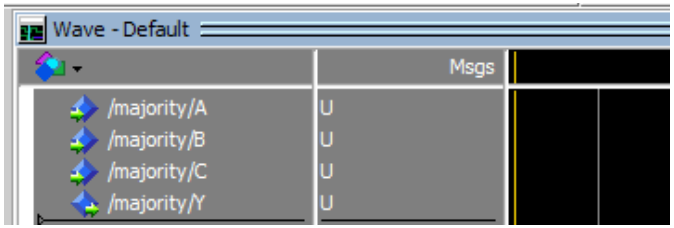
- **Passo 5**: Simulação no ***ModelSim***
  - Para abrir o simulador **Modelsim** use **Tools -> Run Simulation Tool -> RTL Simulation** no **Quartus II**
  - Dê um duplo clique no módulo a ser simulado no grupo **work**:



  - Selecione os sinais de interesse (**A**, **B**, **C** e **Y**) na janela **Objects**, e os adicione à simulação usando a opção **Add Wave**:

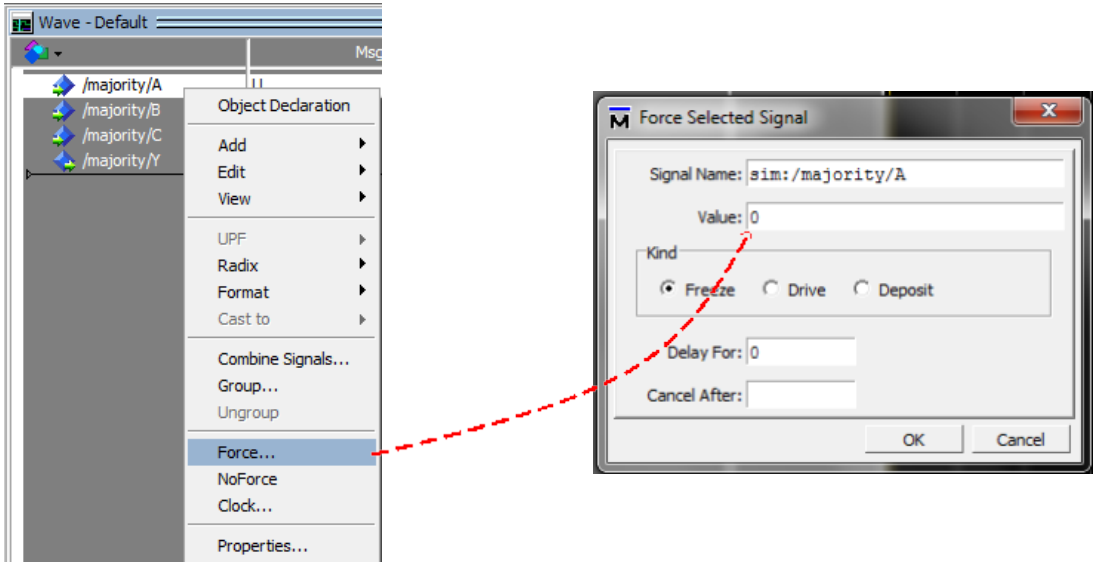


  - Com isso, os sinais de interesse deverão aparecer na janela **Wave**:



Tarefa

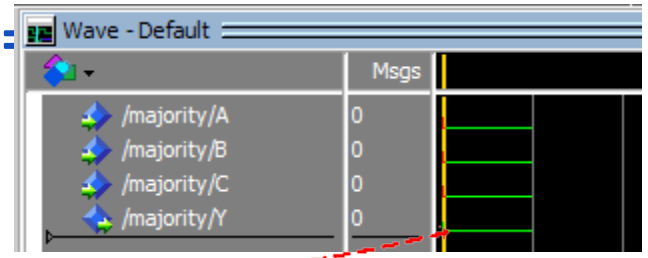
- **Passo 5**: Simulação no ***ModelSim***
  - Utilize a opção ***Force*** para atribuir valores às entradas **A**, **B** e **C** do circuito:



  - Atribua inicialmente os valores **A = B = C = 0** e, em seguida, clique em **Simulate -> *Run*** (ou simplesmente pressione **F9**) para simular o circuito com tais valores nas entradas.

Tarefa

- **Passo 5**: Simulação no ***ModelSim***
  - Como resultado, você deverá observar uma linha verde em nível lógico baixo para a saída **Y**, uma vez que, para o circuito considerado, **Y = 0** para **A =**



  - Simule então agora o circuito para todas as combinações possíveis de valores das entradas e complete a **tabela verdade**:

A B C	Y
0 0 0	0
0 0 1	
0 1 0	
0 1 1	
1 0 0	

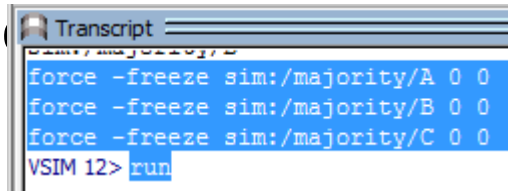
Tarefa

- **Passo 5**: Simulação no ***ModelSim***

  - Finalmente, compare a **tabela verdade obtida** com a **tabela verdade esperada para o seu circuito** e verifique se ele está funcionando corretamente.

Tarefa

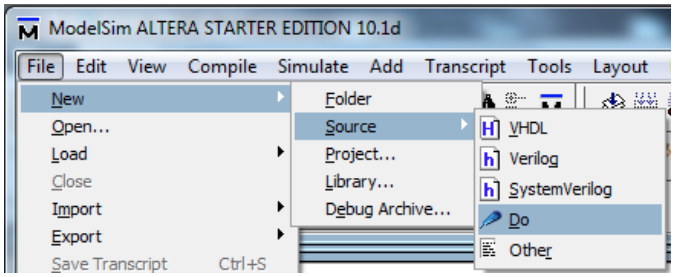
- **Passo 6**: Automatizando a Simulação
  - O procedimento de simulação pode ser automatizado com a criação de um **script de simulação (testbech)**.
  - Para tal, force **A = B = C = 0**, rode sua simulação (pressionando **F9**) e, na janela **Transcript**, você poderá observar a sequencia de quatro comandos utilizada para forçar valores e ro



  - Para criar um **script** a partir de tais comandos, inicialmente você deve selecionar tais comandos e copiá-los (**Ctrl+C**).

Tarefa

- **Passo 6**: Automatizando a Simulação
  - Cria agora um arquivo de **script** usando **File -> New -> Source -> Do**



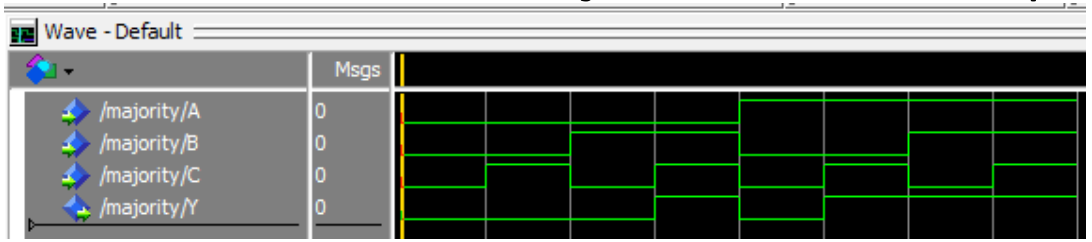
  - Cole os comandos copiados e modifique-os para que a simulação seja executada com diferentes combinações de valores para **A**, **B** e **C**:

C:/altera/15.0/Untitled-1.do - Default *

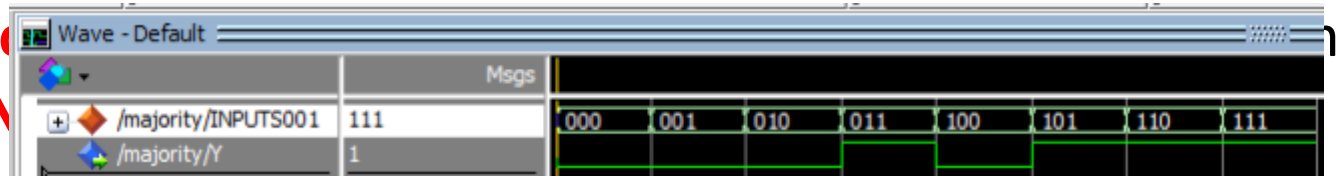
```
Ln#
 1  force -freeze sim:/majority/A 0 0
 2  force -freeze sim:/majority/B 0 0
 3  force -freeze sim:/majority/C 0 0
 4  run
 5  force -freeze sim:/majority/A 0 0
 6  force -freeze sim:/majority/B 0 0
 7  force -freeze sim:/majority/C 1 0
 8  run
 9  force -freeze sim:/majority/A 0 0
10  force -freeze sim:/majority/B 1 0
11  force -freeze sim:/majority/C 0 0
12  run
13  force -freeze sim:/majority/A 0 0
14  force -freeze sim:/majority/B 1 0
15  force -freeze sim:/majority/C 1 0
16  run
```

Tarefa

- **Passo 6**: Automatizando a Simulação
  - Salve seu script (por exemplo, como **sim1.do**) e, para executá-lo, digite o seguinte comando na janela **Transcript**:
    do sim1.do
  - Como resultado, sua simulação deverá rodar para todas as comb



  - Para facilitar a visualização dos resultados, você pode selecionar os três sinais de entrada (**A**, **B** e **C**) e usar a opção
    C ne
    IN



Tarefa

- **Passo 6**: Automatizando a Simulação

  - Faça o *print* de tela da simulação mostrando **claramente** que as 8 combinações possíveis de ABC estão funcionando corretamente como uma operação maioria.

  - Com o intuito de não acontecer colas, o marcador amarelo deverá estar colocado na combinação de entrada X, sendo X o ultimo número da matrícula do aluno. Caso seja 8 ou 9 usará 6 e 7, respectivamente. Se o trabalho é feito em dupla, será usada a matrícula de maior valor.

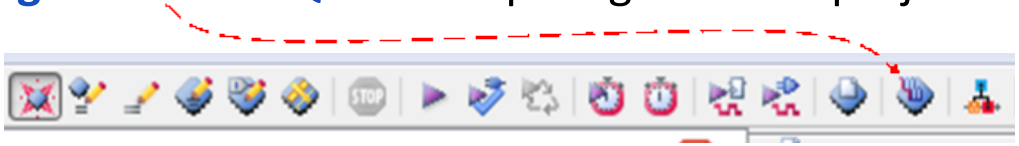
Introdução

Tarefa

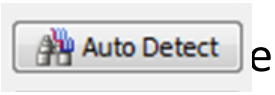
Execução na placa DE2

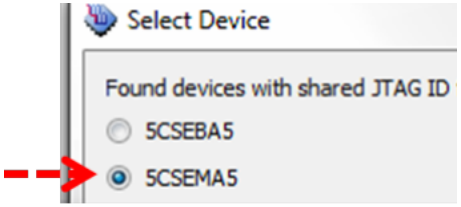
Testando no DE1-SoC

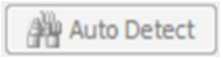
- **Gravação no DE1-SoC**
  - Com o kit **DE1-SOC** ligado à tomada e ao computador, acesse **Tools > Programmer** no **Quartus II** para gravar seu projeto.

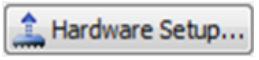


  - Com o **Programmer** aberto:

    1) Clique em 

e selecione a 2ª opção: 

    Caso **Auto Detect** esteja desativado (

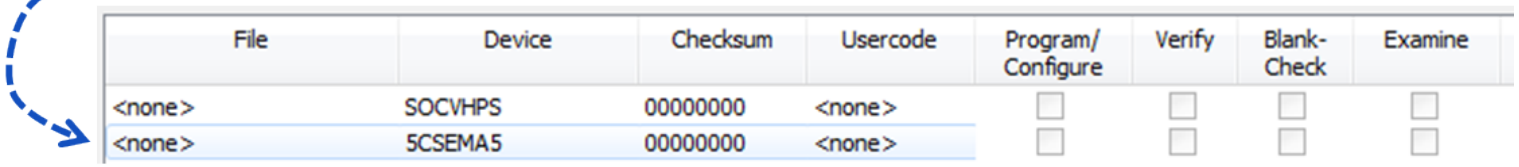
), clique em 

e selecione **DE1-SoC** como hardware a ser utilizado.

Testando no DE1-SoC

- **Gravação no DE1-SoC**

  2) A seguir, clique com o **botão da direita** do mouse no **segundo elemento** da lista de arquivos e escolha ***Change File***.



  3) Selecione o arquivo com extensão **.sof** da sub-pasta **output_files** da pasta do seu projeto.

  4) Marque a caixa **Program/Configure** e clique em 

.

  5) Pronto, agora é só testar o seu projeto no **DE1-SoC**.